

Website screenshot

===== Customer
=====













ออเดอร์ที่สั่ง

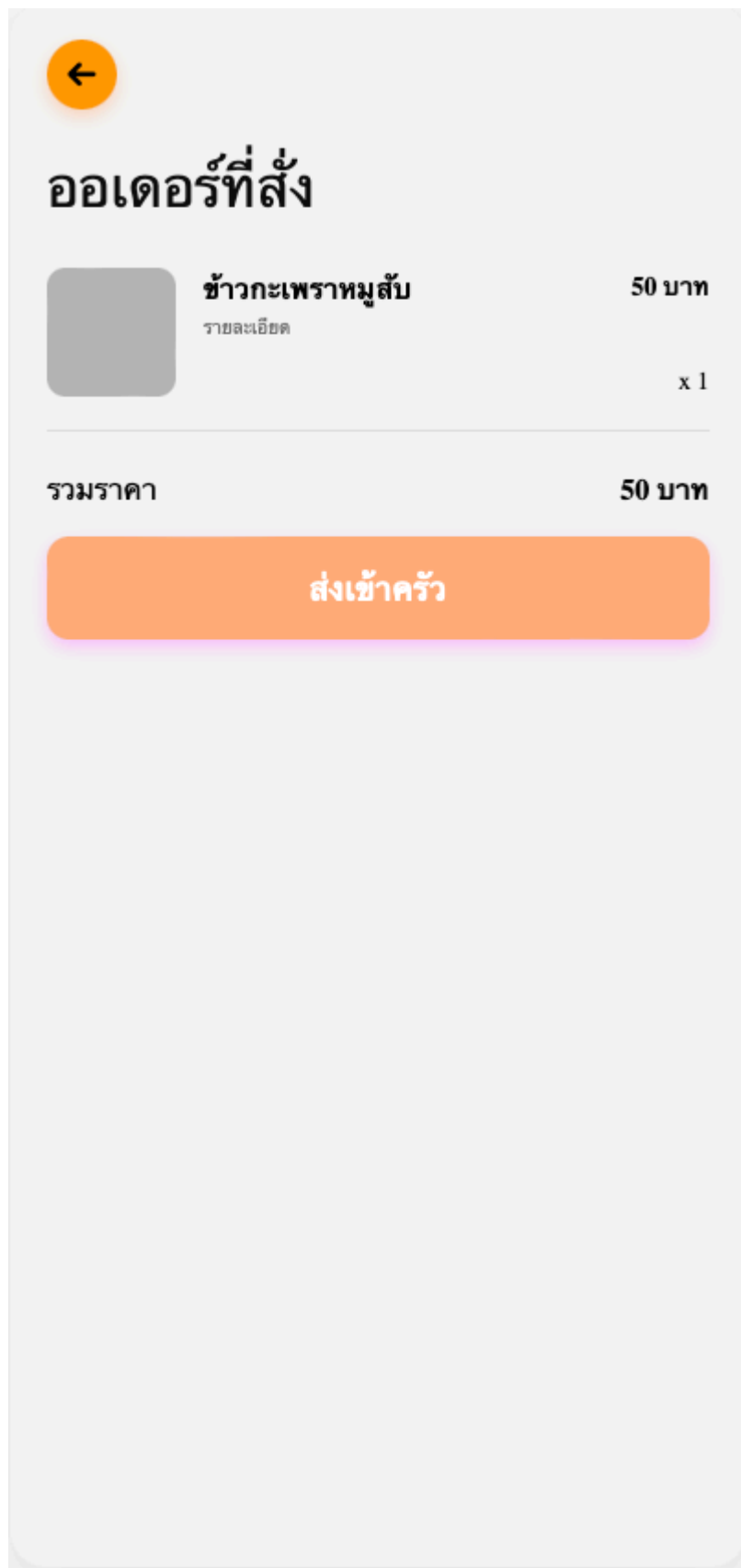
ออเดอร์ของคุณยังว่างเปล่า

[ไปเลือกเมนูอาหาร](#)

รวมราคา

0 บาท

ชำระเงิน





ออเดอร์ที่สั่ง



ข้าวกะเพราหมูสับ

รายละเอียด

50 บาท

x 1

รวมราคา

50 บาท

ส่งเข้าครัว

ต้องการส่งรายการอาหารไปยังห้องครัว?

ยืนยัน

ยกเลิก



ออเดอร์ที่สั่ง



ข้าวกะเพราหมูสับ

รายละเอียด

50 บาท

1

รวมราคา

50 บาท

ชำระเงิน



ออเดอร์ที่สั่ง



ข้าวกะเพราหมูสับ
รายละเอียด

50 บาท

1

เลือกช่องทางการชำระเงิน

 ชำระด้วยเงินสด

 สแกน QR โค้ด

ยกเลิก

รวมราคา

50 บาท

ชำระเงิน

===== BlackOffice
=====

11 / 22

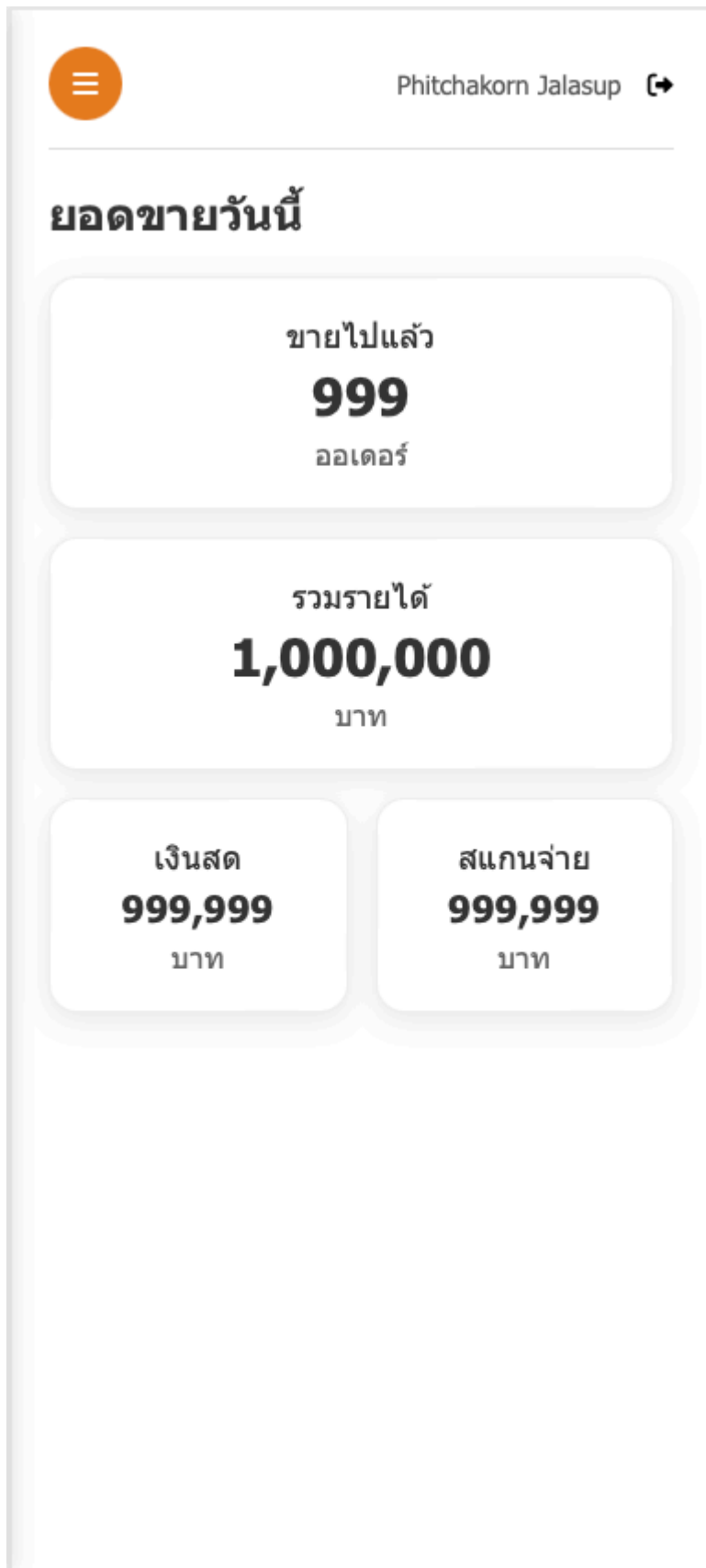
TamSang

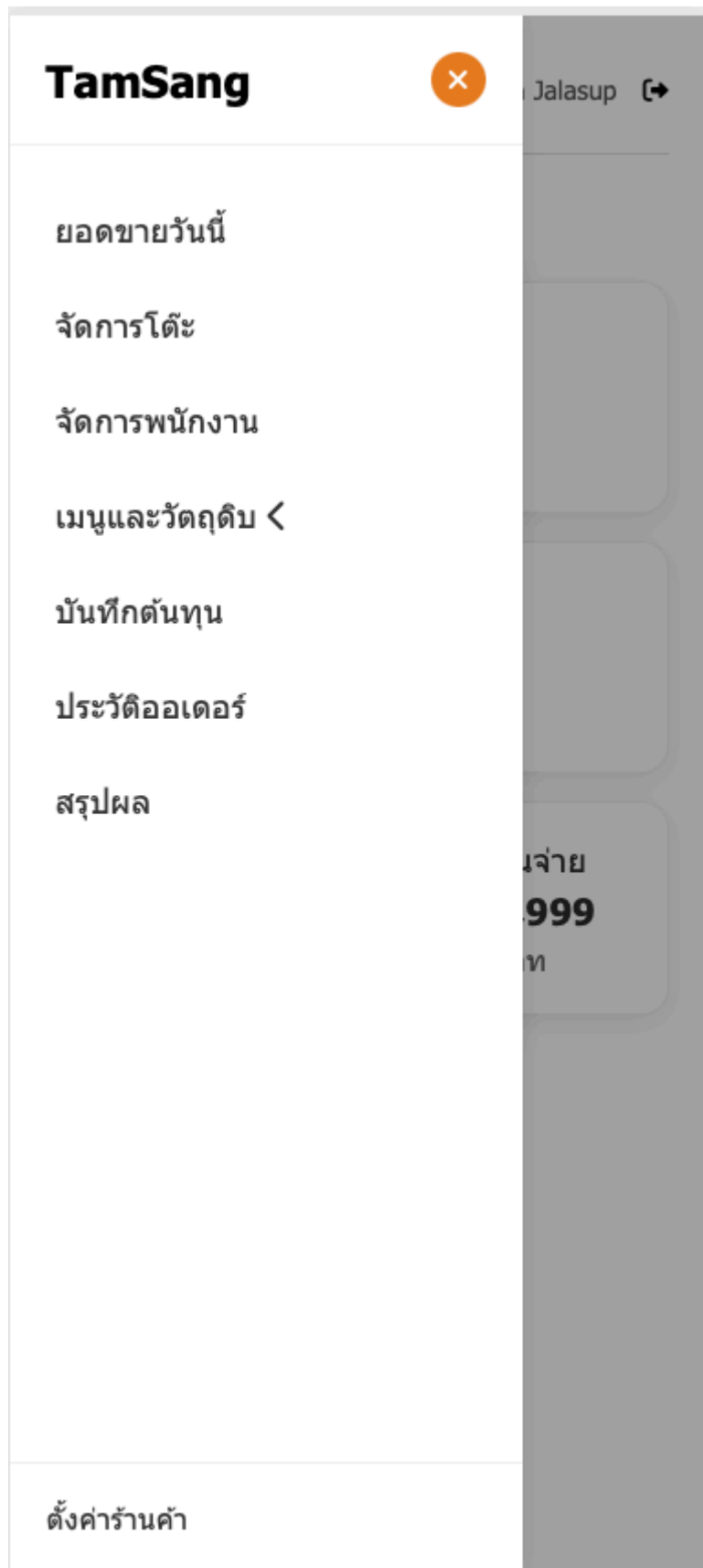
เข้าสู่ระบบ

หรือ



เข้าสู่ระบบด้วย Google



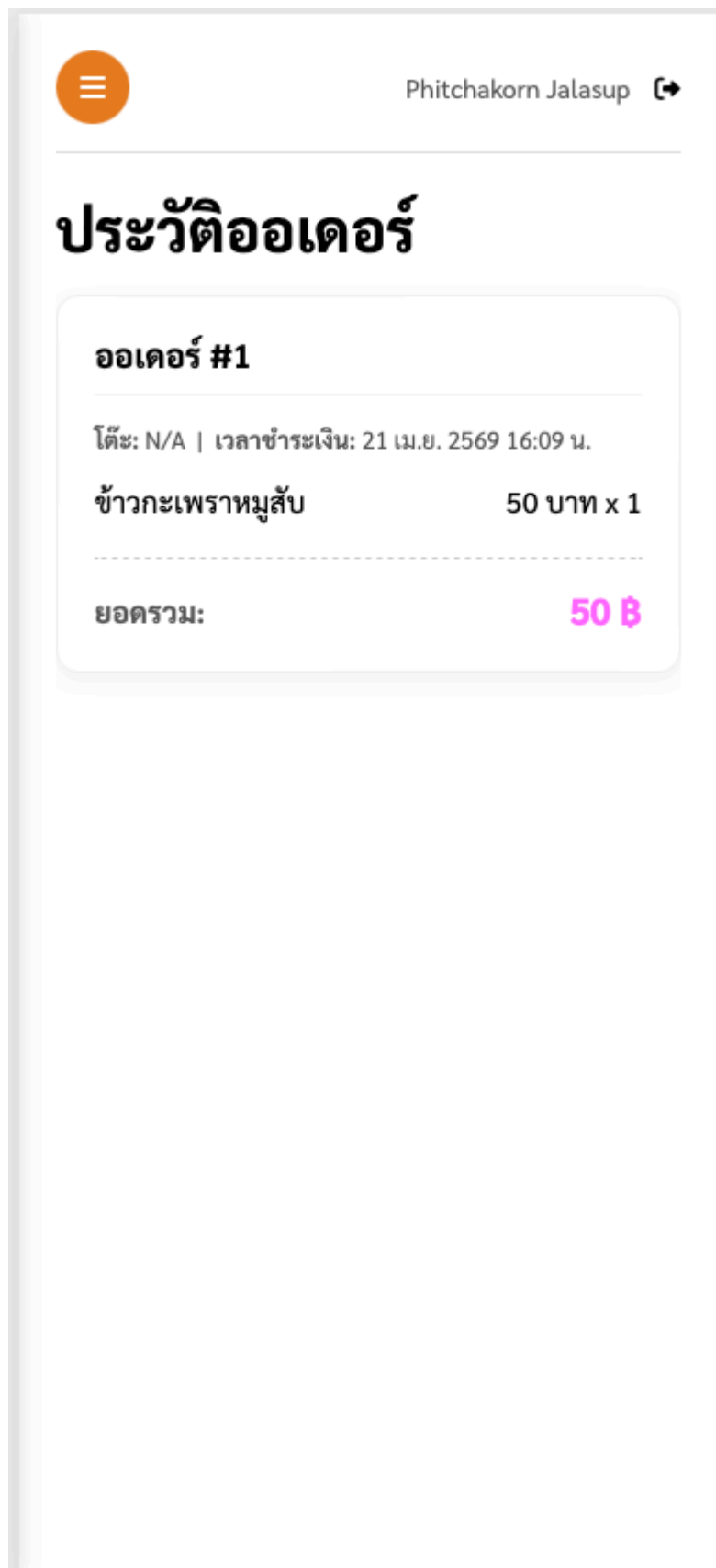




Phitchakorn Jalsup

ประวัติออเดอร์

ยังไม่มีประวัติการสั่งซื้อ



UI testcases

▼ tamsang-flow.spec.js		
✓ TamSang User Flow & Logic Tests › 1. Cart items should persist after reopening the browser	chromium	1.0s
tamsang-flow.spec.js:6		
✓ TamSang User Flow & Logic Tests › 2. Should scroll to category when clicked	chromium	616ms
tamsang-flow.spec.js:35		
✓ TamSang User Flow & Logic Tests › 3. Admin sidebar should be visible after login	chromium	701ms
tamsang-flow.spec.js:47		
✓ TamSang User Flow & Logic Tests › 4. Should redirect unauthorized user to login page	chromium	477ms
tamsang-flow.spec.js:66		
✓ TamSang User Flow & Logic Tests › 5. Order should appear in History after checkout	chromium	3.2s
tamsang-flow.spec.js:73		
✓ TamSang User Flow & Logic Tests › 1. Cart items should persist after reopening the browser	firefox	1.7s
tamsang-flow.spec.js:6		
✓ TamSang User Flow & Logic Tests › 2. Should scroll to category when clicked	firefox	1.2s
tamsang-flow.spec.js:35		
✓ TamSang User Flow & Logic Tests › 3. Admin sidebar should be visible after login	firefox	1.3s
tamsang-flow.spec.js:47		
✓ TamSang User Flow & Logic Tests › 4. Should redirect unauthorized user to login page	firefox	397ms
tamsang-flow.spec.js:66		
✓ TamSang User Flow & Logic Tests › 5. Order should appear in History after checkout	firefox	3.4s
tamsang-flow.spec.js:73		
✓ TamSang User Flow & Logic Tests › 1. Cart items should persist after reopening the browser	webkit	2.0s
tamsang-flow.spec.js:6		
✓ TamSang User Flow & Logic Tests › 2. Should scroll to category when clicked	webkit	1.1s
tamsang-flow.spec.js:35		
✓ TamSang User Flow & Logic Tests › 3. Admin sidebar should be visible after login	webkit	1.2s
tamsang-flow.spec.js:47		
✓ TamSang User Flow & Logic Tests › 4. Should redirect unauthorized user to login page	webkit	300ms
tamsang-flow.spec.js:66		
✓ TamSang User Flow & Logic Tests › 5. Order should appear in History after checkout	webkit	3.4s
tamsang-flow.spec.js:73		

profiling

===== Static profiling
=====

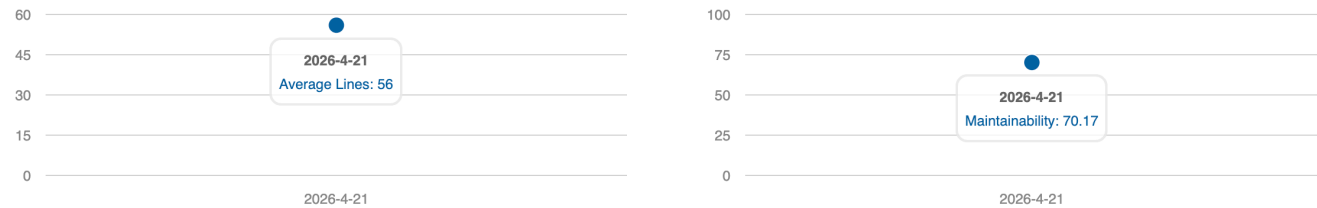
Summary

Total/Average Lines ⓘ

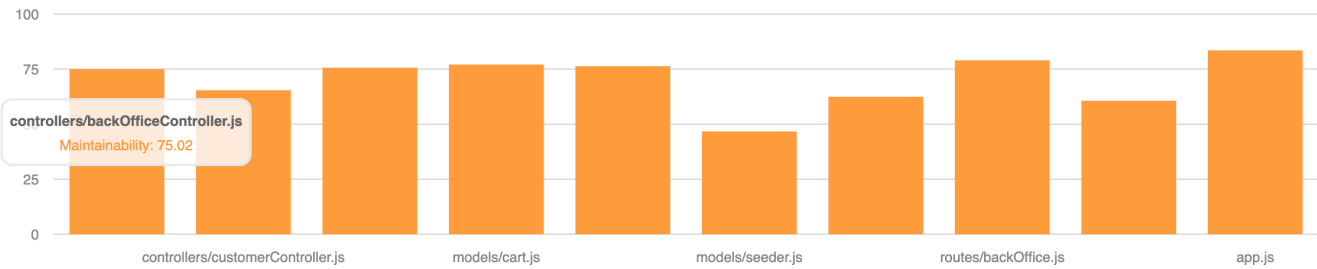
566 / 56

Average Maintainability ⓘ

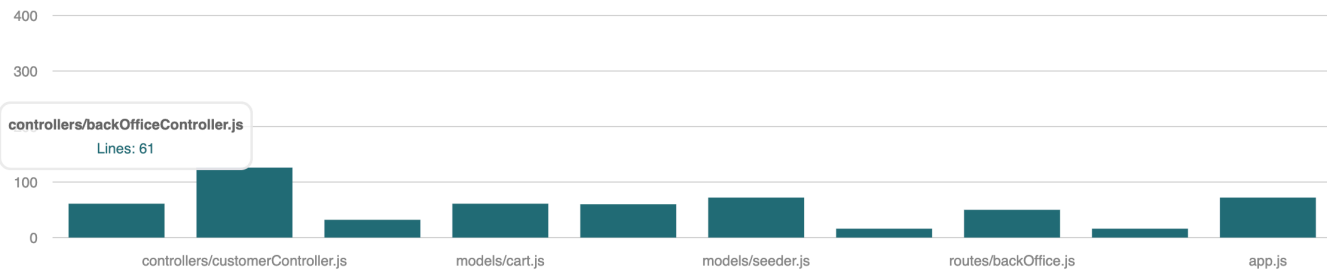
70.17



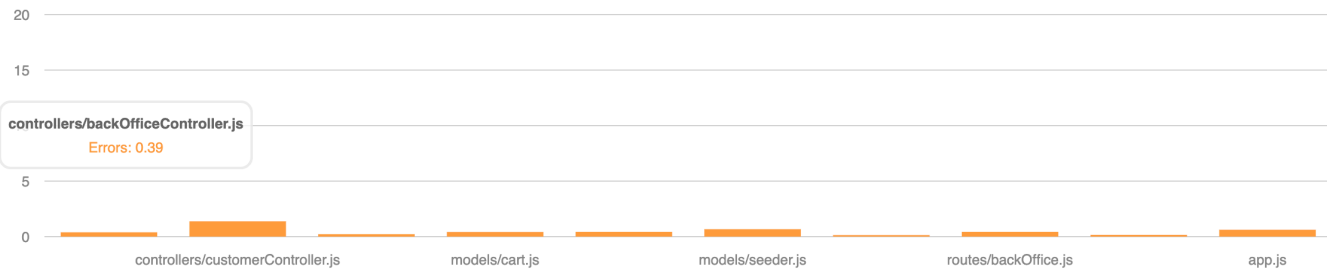
Maintainability ⓘ



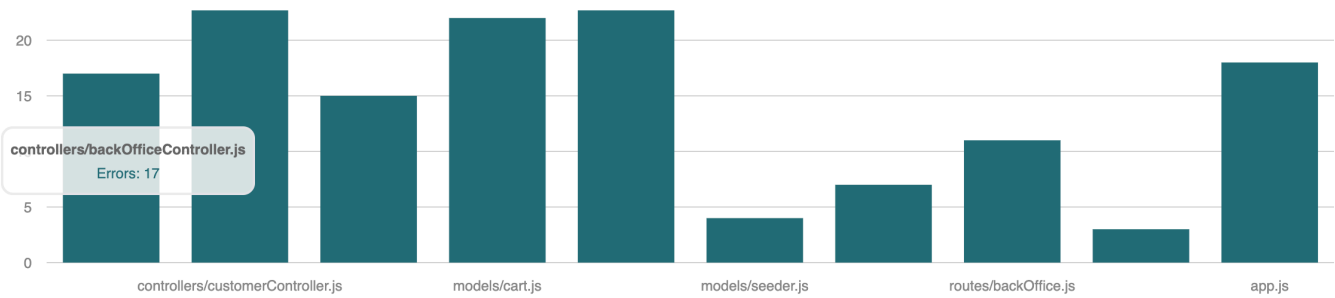
Lines of code ⓘ



Estimated errors in implementation ⓘ



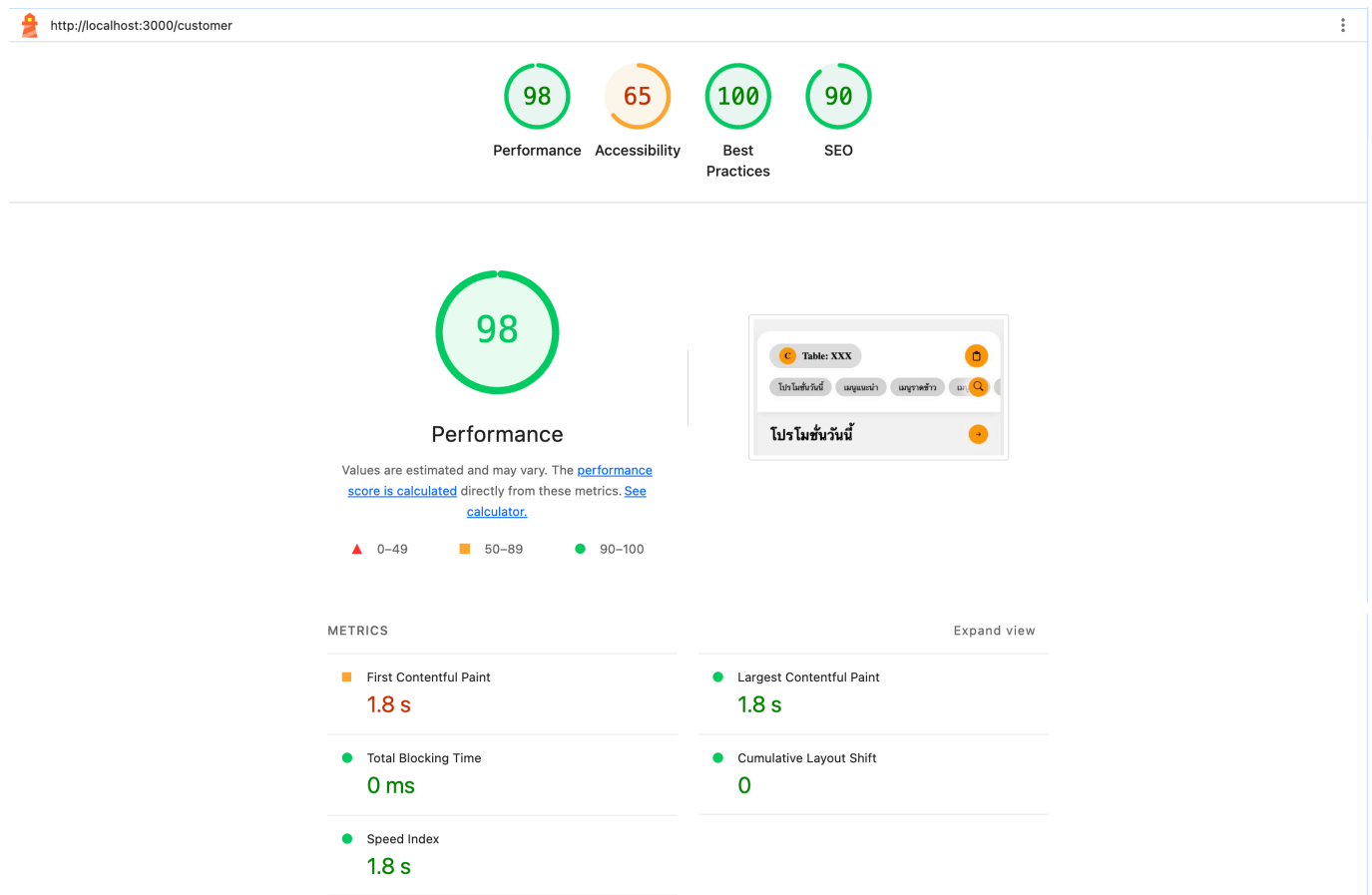
Lint errors



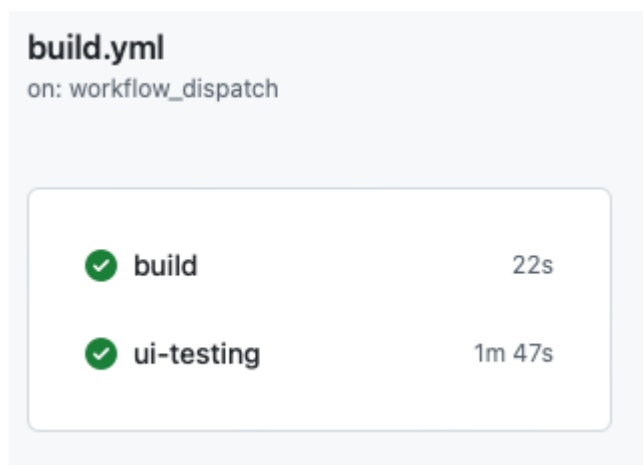
Files

	complexity	sloc	est errors	lint errors
controllers/backOfficeController.js	5	61	0.39	17
controllers/customerController.js	14	126	1.38	51
models/admin.js	4	32	0.22	15
models/cart.js	6	61	0.42	22
models/order.js	4	60	0.43	28
models/seeder.js	1	72	0.67	4
routes/api.js	2	16	0.14	7
routes/backOffice.js	10	50	0.43	11
routes/customer.js	1	16	0.16	3
app.js	3	72	0.62	18

===== Dynamic profiling
=====



อธิบายการทำ CI/CD ที่ใช้ในการทำ product



1. Job: build (Unit Testing)

ตรวจสอบ "ตรรกะภายใน" ของโค้ด (Logic/Models) ว่าคำนวณถูกไหม เชื่อมต่อ Database จำลองได้หรือไม่
เป้าหมาย: รันไฟล์ในโฟลเดอร์ test/ (พวก *.test.js)

กระบวนการ: ติดตั้ง Library แล้วใช้ Jest รันสคริปต์ทดสอบโดยตรง ไม่ต้องเปิดหน้าเว็บ ไม่ต้องรัน Server จริง

2. Job: ui-testing (End-to-End Testing)

ตรวจสอบ "ประสบการณ์ผู้ใช้" (User Flow) โดยจำลองการใช้งานจริงผ่าน Browser

เป้าหมาย: รันไฟล์ในโฟลเดอร์ tests/ (พวก *.spec.js)

กระบวนการ:

1. ติดตั้ง Browser (Chromium/Firefox)

2. สั่ง npm start & เพื่อเปิด Server ของแอปขึ้นมาจริงๆ ในเครื่องจำลอง
3. ใช้ wait-on รอจนหน้าเว็บ localhost:3000 ตื่นขึ้นมาพร้อมใช้งาน
4. ใช้ Playwright เปิด Browser เข้าไปกดปุ่ม, กรอกฟอร์ม และเช็คผลลัพธ์บนหน้าจอ

กระบวนการทำงาน

- process:
 - การออกแบบ UX/UI
 - การพัฒนาโค้ด
 - การทดสอบระบบแบบอัตโนมัติ (Automated Testing): เพิ่มขั้นตอนการตรวจสอบโค้ดทุกครั้งก่อนส่งงาน เพื่อป้องกันข้อผิดพลาด (Bugs) ในระดับ Logic และ UI
 - การวิเคราะห์คุณภาพซอฟต์แวร์ (Static Code Analysis): เพิ่มการตรวจสอบโครงสร้างโค้ดและความซับซ้อน เพื่อให้โค้ดอ่านง่ายและบำรุงรักษาได้ในระยะยาว
 - การทำ Continuous Integration (CI): สร้างระบบที่ช่วยรันเทสและตรวจสอบความพร้อมของโปรเจกต์โดยอัตโนมัติบน Cloud
 - การทำ Continuous Delivery (CD): ช่วยในการ deploy product
- methods:
 - สร้าง Prototype เพื่อกำหนด Flow การใช้งานและ Mood & Tone ของแอปพลิเคชัน
 - Unit Testing: ใช้การเขียนสคริปต์ทดสอบฟังก์ชันแยกเป็นส่วนๆ (เช่น ระบบ Login, การคำนวณราคาสั่งอาหาร) เพื่อเช็คความถูกต้องของ Logic
 - End-to-End (E2E) UI Testing: จำลองพฤติกรรมผู้ใช้จริง (User Flow) ตั้งแต่การเข้าหน้าเว็บ เลือกเมนู จนถึงการชำระเงิน
 - Parallel Execution: แยกการทำงานของ Test ออกเป็นหลายส่วน (Jobs) และรันพร้อมกันเพื่อลดเวลาในการรอผล
 - Environment Mocking: การจำลองค่าตัวแปรสภาพแวดล้อม (Env) และการ Bypass ระบบบางส่วนที่ไม่ได้ทดสอบ (เช่น Google OAuth) เพื่อให้เทสเดินหน้าต่อไปได้
- tools:
 - Figma (ใช้ในการทำ Design System, Wireframing และ Prototyping)
 - Node.js: ใช้เป็น Runtime หลักในการเขียนโปรแกรม บน Visual Studio Code
 - GitHub Actions: ใช้เป็นหัวใจหลักในการทำ CI/CD เพื่อสร้าง Workflow อัตโนมัติ
 - Jest: เครื่องมือสำหรับรัน Unit Test เพื่อตรวจสอบความถูกต้องของ Model และ Controller
 - Playwright: เครื่องมือสำหรับรัน UI Test โดยจำลอง Browser (Chromium, Firefox) มาทดสอบหน้าเว็บ
 - Wait-on: ยูทิลิตี้สำหรับรอให้ Server (localhost:3000) ตื่นขึ้นมาก่อนที่เทสจะเริ่มรัน
 - Plato: เครื่องมือสำหรับทำ Static Profiling เพื่อสร้าง Report วิเคราะห์ความซับซ้อนของ JavaScript

Final Retrospective

<https://youtu.be/N2Z9ZR1CeUM>

ทิมมพร ท่วมแก้ว 161

พิชชากร จลาสุก 167

อามีน สาแม 532

ปัญหาที่พบในการทำงาน:

1. การจัดการเวลา: สมาชิกในทีมประสบปัญหาการบริหารเวลาที่ยังไม่ดีพอ โดยมีการผลัดวันประกันพรุ่งเพราะคิดว่างานใช้เวลาทำไม่นาน ทำให้งานไปกระจุกตัวและหนักในช่วงท้ายโปรเจกต์ ประกอบกับมีงานจากวิชาอื่นๆ เข้ามาแทรกพร้อมกัน
2. การแบ่งงานและการะงาน: การกระจายงานยังทำได้ไม่เหมาะสม ทำให้ภาระงานและปัญหาในการแก้โค้ดส่วนใหญ่ตกไปอยู่ที่คุณพิชชากรเพียงคนเดียว โดยที่คุณทิมมพรและคุณอามีนรู้สึกว่าตนเองไม่ได้ช่วยเหลือนงานมากเท่าที่ควร
3. ความเข้าใจโค้ดและการสื่อสาร: มีปัญหาการสื่อสารที่ทำให้เข้าใจไม่ตรงกัน โดยเฉพาะเรื่องการตั้งค่าตัวแปรและการเก็บข้อมูล ทำให้เกิดข้อผิดพลาดและต้องนำมาแก้ไข นอกจากนี้คุณทิมมพรรู้สึกว่ามีความเวลาในการทำความเข้าใจโค้ดน้อยเกินไป ทำให้ผลงานออกมายังไม่มีคุณภาพเท่าที่ควร
4. ปัญหาด้านอุปกรณ์: คุณทิมมพรมีปัญหาเกี่ยวกับอุปกรณ์อิเล็กทรอนิกส์ที่ใช้ทำงาน
5. การพึ่งพาผู้อื่น: คุณอามีนมองว่าตนเองยังพึ่งพาคนอื่นในการทำงานมากเกินไป ไม่ค่อยได้ลงมือทำด้วยตนเอง

แนวทางแก้ไขและปรับปรุงสำหรับโปรเจกต์ถัดไป:

1. บริหารจัดการและติดตามงานให้ดีขึ้น: ทีมตกลงที่จะกระจายงานให้เหมาะสม บริหารจัดการเวลาและติดตามความคืบหน้าของงานให้ดีขึ้น เพื่อรักษาสมดุล (Balance) ไม่ให้งานไปหนักตอนท้ายโปรเจกต์อีก
2. พัฒนาทักษะส่วนตัว: คุณทิมมพรตั้งเป้าหมายที่จะเร่งศึกษาหาความรู้ด้านการเขียนโค้ดให้มากขึ้น เพื่อจะได้ช่วยแบ่งเบาภาระของคุณพิชชากรได้ คุณอามีนจะพยายามหาทักษะเพิ่มเติมเพื่อให้สามารถช่วยเหลือตัวเองและทำงานเดี่ยวได้มากขึ้น โดยไม่ต้องรอคนอื่น
3. เตรียมความพร้อมด้านอุปกรณ์: คุณทิมมพรวางแผนจะเก็บเงินซื้ออุปกรณ์อิเล็กทรอนิกส์ใหม่ที่มีประสิทธิภาพมากขึ้น เพื่อป้องกันไม่ให้เกิดปัญหาในการทำงานร่วมกันในอนาคต